

Министерство цифрового развития, связи и массовых коммуникаций РФ
Федеральное государственное бюджетное образовательное учреждение высшего
образования

«Сибирский государственный университет
телекоммуникаций и информатики» (СибГУТИ)

Кафедра телекоммуникационных систем и вычислительных средств

Практическое задание № 4

Изучение корреляционных свойств последовательностей, используемых для
синхронизации в сетях мобильной связи.

Выполнил: студент 3 курса

гр. ИА-331

Помелова А.В.

Новосибирск 2025

ЦЕЛЬ ЗАНЯТИЯ

Получить представление о том, какие существуют псевдослучайные двоичные последовательности, какими корреляционными свойствами они обладают и как используются для синхронизации приемников и передатчиков в сетях мобильной связи.

КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Псевдослучайные последовательности (PN-sequences) – бинарные последовательности (1/0 или +1/-1), которые кажутся случайными, но детерминированы.

Свойства PN-последовательностей:

- Сбалансированность: число единиц и число нулей на любом интервале последовательности должно отличаться не более чем на одну.
- Цикличность: циклом в данном случае является последовательность бит с одинаковыми значениями. В каждом фрагменте псевдослучайной битовой последовательности примерно половину составляли циклы длиной 1, одну четверть – длиной 2, одну восьмую – длиной 3 и т.д.
- Корреляция: корреляция оригинальной битовой последовательности с ее сдвинутой копией должна быть минимальной. Автокорреляция этих последовательностей – практически дельта-функция во временной области, а в частотной области – это константа.

Применение в мобильной связи:

- Синхронизация приёмника и передатчика.
- Расширение спектра.
- Оценка BER (вероятность битовой ошибки).

Примеры PN-последовательностей: M-последовательности, коды Голда, коды Касами, коды Уолша-Адамара.

Коды Голда формируются суммированием по модулю 2 двух M-последовательностей одинаковой длины.

ЗАДАНИЕ ДЛЯ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

В рамках данной работы студенты должны научиться формировать псевдослучайные битовые последовательности (коды Голда), изучить их автокорреляционные и взаимокорреляционные свойства.

1) Напишите программу на языке C/C++ для генерации последовательности Голда, используя схему, изображенную на рисунке 4.4, если ваша группа с четным номером и 4.5 – если с нечетным, и порождающие полиномы x и y , при этом x – это ваш порядковый номер в журнале в двоичном формате (5 бит), а y – это $x+7$ (5 бит). Например, ваш номер 22, значит:
 $x = 1\ 0\ 1\ 1\ 0$, $y = 1\ 1\ 1\ 0\ 1$.

Мой номер – 19

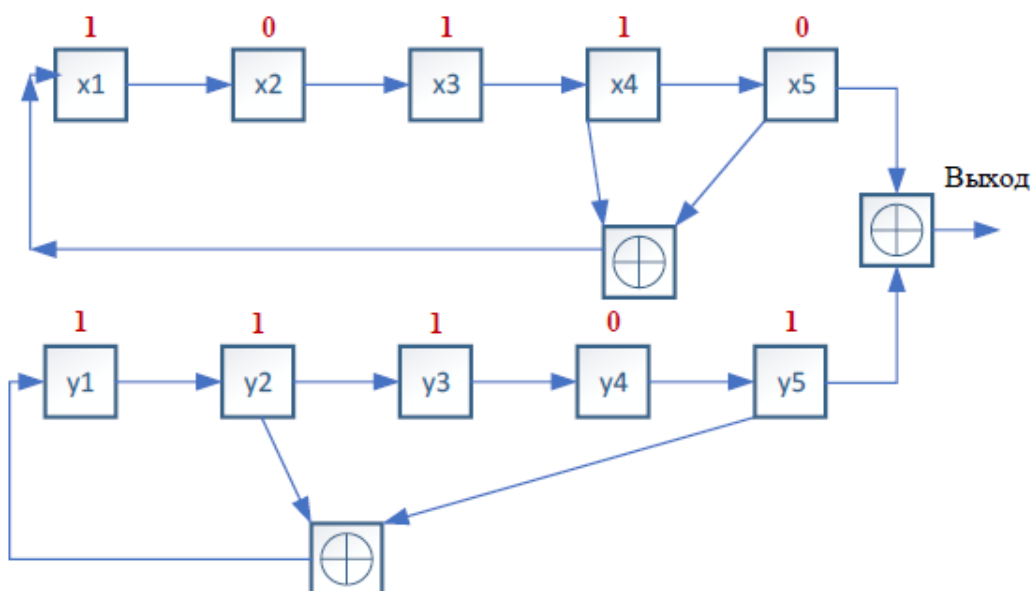


Рис. 4.5. Генерация последовательности Голда (вариант для нечетного номера группы)

2) Выведите получившуюся последовательность на экран.

3) Сделайте поэлементный циклический сдвиг последовательности и посчитайте автокорреляцию исходной последовательности и сдвинутой. Сформируйте таблицу с битовыми значениями последовательностей, в последнем столбце которой будет вычисленное значение автокорреляции, как показано в примере ниже.

Сдвиг	бит 1	бит 2	бит 3	Автокорреляция
0	1	1	0	1
1	0	1	1	-1/3
2	0	1	1	-1/3
3	1	1	0	1

4) Сформируйте еще одну последовательность Голда, используя свою схему (рис 4.5), такую что $x = x + 1$, а $y = y - 5$.

5) Вычислите значение взаимной корреляции исходной и новой последовательностей и выведите в терминал.

6) Прodelайте шаги 1-5 в Matlab. Используйте функции `xcorr()` и `autocorr()` для вычисления соответствующих корреляций. Сравните результаты, полученные в Matlab и C/C++.

7) Выведите на график в Matlab функцию автокорреляции в зависимости от величины задержки (lag).

8) Составьте отчет. Отчет должен содержать титульный лист, содержание, цель и задачи работы, теоретические сведения, исходные данные, этапы выполнения работы, сопровождаемые скриншотами и графиками, демонстрирующими успешность выполнения, и промежуточными выводами, результирующими таблицами, ответы на контрольные вопросы, и заключение и **ссылка в виде QR-кода на репозиторий с кодом (git)**.

ВЫПОЛНЕНИЕ ПРАКТИЧЕСКОЙ РАБОТЫ

1) Напишите программу на языке C/C++ для генерации последовательности Голда

```
void generate_gold(int x[5], int y[5], int gold[31]) {
    for (int i = 0; i < 31; i++) {
        gold[i] = x[4] ^ y[4];           // выход = x5 XOR y5
        int new_x1 = x[3] ^ x[4];       // x1 = x4 XOR x5
        int new_y1 = y[1] ^ y[4];       // y1 = y2 XOR y5

        for (int j = 4; j > 0; j--) {
            x[j] = x[j-1];
            y[j] = y[j-1];
        }
        x[0] = new_x1;
        y[0] = new_y1;
    }
}
```

2) Выведите получившуюся последовательность на экран.

Последовательность Голда (31 бит):

1 0 0 1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 1 0

```
Последовательность Голда (31 бит):
1 0 0 1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 1 0
pomelova@FEIHOA:/mnt/c/Users/pomel/Desktop/Study/Основы СМС/Лаба4$
```

3) Сделайте поэлементный циклический сдвиг последовательности и посчитайте автокорреляцию исходной последовательности и сдвинутой. Сформируйте таблицу с битовыми значениями последовательностей

```
printf("\n\nСдвиг");
for (int i = 1; i <= N; i++) {
    printf(" | %d", i);
}
printf(" | Автокорреляция\n");
for (int i = 0; i < N; i++) printf("-----");
printf("\n");

for (int tau = 0; tau < N; tau++) {
    double c = 0.0;
    printf("%4d", tau);

    for (int i = 0; i < N; i++) {
```

```

    int j = (i - tau + N) % N;          // правый сдвиг на tau
    int shifted_bit = gold1[j];
    printf(" | %d", shifted_bit);
    c += (double)(gold1[i] * shifted_bit);
}

double R = (sum_sq > 0) ? c / sum_sq : 0.0;
printf(" | %.3f\n", R);
}

```

Сдвиг | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 |
 27 | 28 | 29 | 30 | 31 | Автокорреляция

0		1		0		0		1		0		1		1		1		0		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		1.000				
1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0.556		
2		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		0		1		1		1		0		0		0		0		1		0		0.667				
3		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		0		1		1		1		0		0		0		0		1		0.556				
4		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		1		0		1		1		1		0		0		0		0		0.667		
5		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		1		0		1		1		1		0		0		0		0.556		
6		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		1		0		1		1		1		0		0		0.611		
7		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		1		0		1		1		1		0		0.611		
8		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		1		0		1		1		0		0.556		
9		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		1		0		1		1		0.500		
10		1		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		1		0		1		0.667		
11		1		1		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		0		1		0.556		
12		1		1		1		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		0		0.444		
13		0		1		1		1		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		0.500		
14		1		0		1		1		1		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		0.556		
15		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		0		1		0		1		1		1		0		1		1		1		1		0.500		
16		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		1		0.500		
17		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0		0.556		
18		1		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		1		0.500		
19		0		1		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		1		0.444		
20		1		0		1		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		1		0		0		1		0		1		0.556		
21		1		1		0		1		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		1		0		0		1		1		0.667		
22		1		1		1		0		1		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		0		1		0		1		0.500		
23		0		1		1		1		0		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		0		1		0		1		0.556		
24		1		0		1		1		1		0		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0		0		1		0		0.611		
25		1		1		0		1		1		1		0		1		1		1		1		1		0		1		1		0		0		0		0		0		0		1		0		1		0		0		0.611		
26		1		1		1		0		1		1		1		1		0		1		1		1		1		0		1		1		0		0		0		0		0		1		0		1		0		0		0.556		
27		0		1		1		1		0		1		1		1		1		0		1		1		1		1		0		1		1		1		0		0		0		0		0		0		1		0		0.667		
28		1		0		1		1		1		0		1		1		1		0		1		1		1		1		1		0		1		1		1		0		0		0		0		0		0		1		0.556		
29		0		1		0		1		1		1		0		1		1		1		0		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		0.667		
30		0		0		1		0		1		1		1		1		0		1		1		1		1		1		1		0		1		1		1		0		0		0		0		0		1		0		1		0.556

Сдвиг	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	Автокорреляция	
0	1	0	0	1	0	1	1	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	0	0	0	1	0	1	0	1	0.000		
1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	1	0.556	
2	1	0	1	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	1	1	0	0	0	0	1	0	1	0	0.667	
3	0	1	0	1	1	0	0	1	0	1	1	1	0	1	1	1	1	1	1	1	0	1	1	1	0	0	0	0	1	0	1	0.556	
4	1	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0	1	0.667	
5	0	1	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0.556	
6	0	0	1	0	1	0	1	0	1	0	0	1	1	1	0	1	1	1	0	1	1	1	1	1	0	1	1	1	0	0	0	0.611	
7	0	0	0	1	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0	0	0	1	0.611	
8	0	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	1	1	0	1	1	0.556	
9	0	0	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	1	1	1	1	0.500	
10	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	1	1	1	1	0	1	1	1	0.667	
11	1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	1	1	1	1	0	1	1	0.556	
12	1	1	1	1	0	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	1	1	1	1	1	1	0.444	
13	0	1	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	1	1	0.500	
14	1	0	1	0	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0.556	
15	1	1	0	1	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	1	0.500
16	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	0.500	
17	1	1	1	1	1	0	1	1	1	1	0	0	0	0	1	0	1	0	0	1	0	0	1	1	1	0	1	1	1	0	1	1	0.556
18	1	1	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	1	1	0	1	1	1	0	1	1	0.500	
19	0	1	1	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	1	0.444	
20	1	0	1	1	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	1	0	0	1	1	1	0	1	1	1	0.556	
21	1	1	0	1	1	1	1	1	1	0	1	1	1	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	0.667	
22	1	1	1	1	0	1	1	1	1	1	0	1	1	1	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	1	0.500	
23	0	1	1	1	1	0	1	1	1	1	1	0	1	1	1	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	1	0.556
24	1	0	1	0	1	1	1	0	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	1	1	1	1	0.611
25	1	1	0	1	1	1	1	0	1	1	1	1	0	1	1	1	0	0	0	0	0	0	1	0	1	0	0	1	0	1	0	1	0.611
26	1	1	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1	0	0	0	0	1	0	1	0	0	1	0	1	0.556	
27	0	1	1	1	1	0	1	1	1	1	0	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	0.667	
28	1	0	1	1	1	1	0	1	1	1	0	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	1	0	0	0.556	
29	0	1	0	1	1	1	1	0	1	1	1	0	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	1	0	0.667	
30	0	0	1	0	1	0	1	1	1	0	1	1	0	1	1	1	1	0	1	1	1	0	0	0	0	1	0	1	0	1	0	1	0.556

rome1ova@FEIHOA: /mnt/c/Users/rome1/Desktop/Study/Основы СМС/Лаба4\$

4) Сформируйте еще одну последовательность Голда, используя свою схему, такую что $x = x + 1$, а $y = y - 5$.

```
int x2[5] = {1, 0, 1, 0, 0}; //20
int y2[5] = {1, 0, 1, 0, 1}; //21
```

С использованием той же функции:

Последовательность Голда 2: 1 0 0 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 0 0 1 1 1 1 1 1

```
Последовательность Голда 1: 1 0 0 1 0 1 1 1 0 1 1 1 0 1 1 1 1 1 0 0 0 0 0 0 1 0 1 0
Последовательность Голда 2: 1 0 0 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 0 0 1 1 1 1 1 1
```

5) Вычислите значение взаимной корреляции исходной и новой последовательностей и выведите в терминал.

```
printf("Сдвиг:   ");
for (int tau = 0; tau < N; tau++) {
    printf("%4d ", tau);
}
printf("\nКорреляция: ");
for (int tau = 0; tau < N; tau++) {
    double c = 0.0;
    for (int i = 0; i < N; i++) {
        int j = (i - tau + N) % N;
        c += (double)(gold1[i] * gold2[j]);
    }
    double R = (d > 0) ? c / d : 0.0;
    printf("%.3f ", R);
}
```

```
printf("\n");
```

Сдвиг:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Корреляция:	0.707	0.530	0.530	0.530	0.530	0.412	0.589	0.471	0.530	0.471	0.648	0.530	0.530	0.530	0.530

15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
0.648	0.530	0.648	0.471	0.589	0.648	0.707	0.589	0.530	0.648	0.471	0.471	0.530	0.412	0.530	0.471

6) Прodelайте шаги 1-5 в Matlab. Используйте функции `xcorr()` и `autocorr()` для вычисления соответствующих корреляций. Сравните результаты, полученные в Matlab и C/C++.

В функциях реализован циклический сдвиг, само значение считается с помощью готовых функций MATLAB

```
function znach = norm_corr(x, y)|
    sum_x = 0;
    sum_y = 0;
    for i = 1:length(x)
        sum_x = sum_x + x(i)^2;
        sum_y = sum_y + y(i)^2;
    end
    c = sum(x .* y);
    znach = c / sqrt(sum_x * sum_y);
end

function ac = auto_corr_xcorr(seq, max_sdvig)
    ac = zeros(1, max_sdvig+1);
    for k = 0:max_sdvig
        seq_sdvig = circshift(seq, [0, k]);
        ac(k+1) = xcorr(seq, seq_sdvig, 0, 'coeff');
    end
end
```

Результаты идентичны с полученными в C

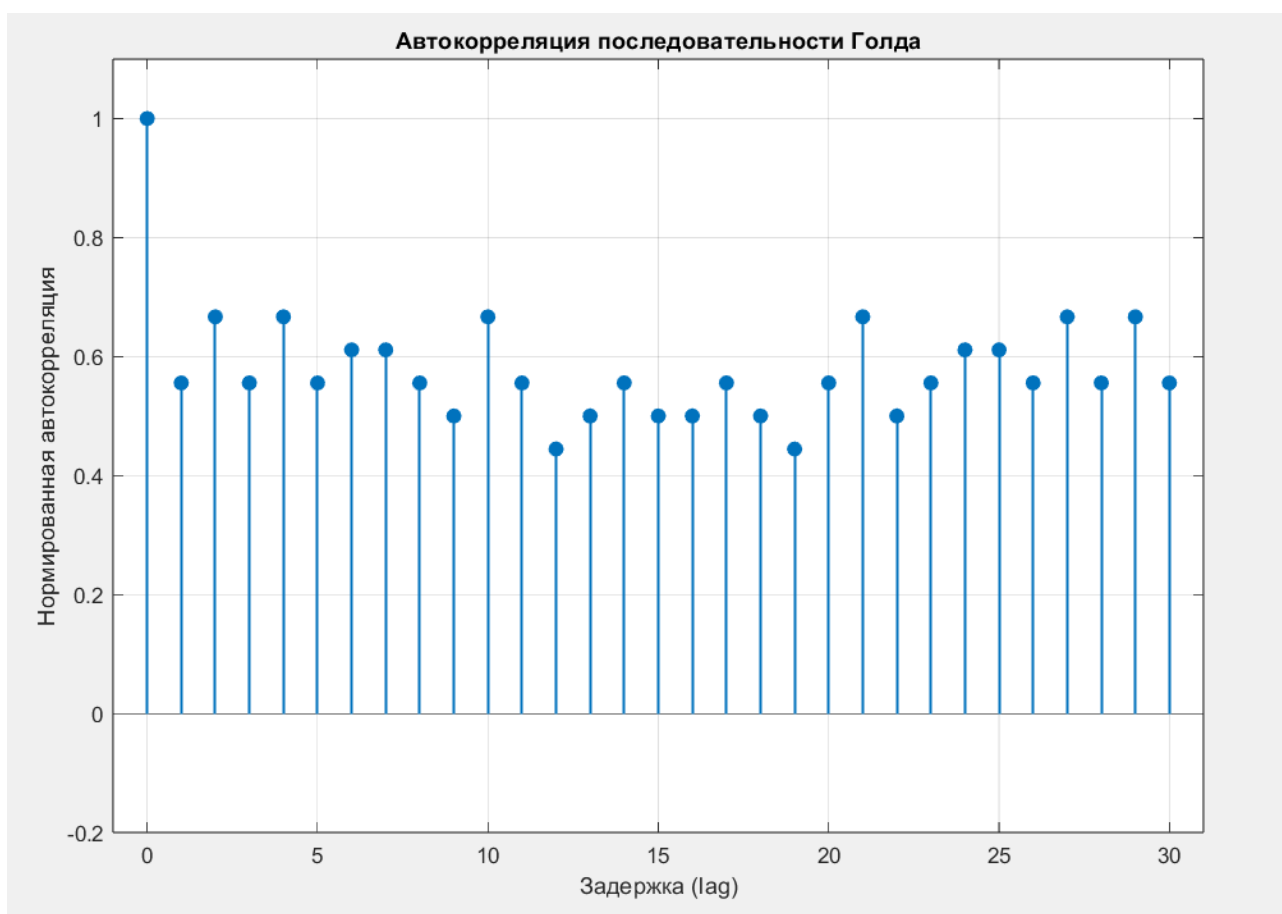
```
>> gold_m
Последовательность Голда 1: 1 0 0 1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 1 0
Последовательность Голда 2: 1 0 0 0 0 1 0 1 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 0 0 1 1 1 1 1 1

Сдвиг:      0      1      2      3      4      5      6      7      8      9      10     11     12     13
Корреляция: 0.707 0.530 0.530 0.530 0.530 0.412 0.589 0.471 0.530 0.471 0.648 0.530 0.530 0.530

14      15      16      17      18      19      20      21      22      23      24      25      26      27      28      29      30
0.530 0.648 0.530 0.648 0.471 0.589 0.648 0.707 0.589 0.530 0.648 0.471 0.471 0.530 0.412 0.530 0.471
```


Сдвиг	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	Автокорреляция		
0	1	0	1	0	0	1	1	0	1	1	1	1	0	1	1	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	1	1.000	
1	0	1	0	1	0	0	1	0	1	1	1	1	0	1	1	0	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0.556	
2	1	1	0	1	1	0	0	1	0	1	1	1	1	0	1	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0	1	0	0.667	
3	0	1	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	1	1	0	1	1	1	0	0	0	0.556	
4	1	1	0	1	1	0	1	0	0	0	1	0	1	1	1	0	1	1	1	1	1	1	1	1	0	1	1	1	1	0	0	0	0.667	
5	0	1	0	1	0	1	0	0	1	0	0	1	0	1	1	1	0	1	1	1	1	1	1	1	0	1	1	1	1	0	0	0	0.556	
6	0	0	0	1	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	1	1	1	1	0	1	1	1	0	0	0.611	
7	0	0	0	0	1	0	1	0	1	0	1	0	0	1	1	0	1	1	1	0	1	1	1	1	1	1	0	1	1	1	1	0	0.611	
8	0	0	0	0	1	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	1	1	1	0	1	1	1	1	1	0	0.556	
9	0	0	0	0	0	1	0	1	0	1	0	1	0	0	1	1	0	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	0.500	
10	1	1	0	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	1	0	1	1	1	0	1	1	1	1	1	1	1	0.667	
11	1	1	1	1	0	0	0	0	0	1	0	1	0	1	0	0	1	1	1	1	0	1	1	1	0	1	1	1	1	1	1	1	0.556	
12	1	1	1	1	1	0	0	0	0	0	1	0	1	0	1	0	0	1	1	1	1	0	1	1	1	0	1	1	1	1	1	1	0.444	
13	0	1	1	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	0	1	1	1	1	0	1	1	1	1	1	1	1	1	0.500	
14	1	1	0	1	1	1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	1	1	0	1	1	1	0	1	1	1	1	0.556	
15	1	1	1	0	1	1	1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	1	1	0.500	
16	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	1	0	1	1	1	1	0.500	
17	1	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0	1	0	1	1	0	1	0	0	1	1	1	0	1	1	1	1	0.556	
18	1	1	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0	1	0	1	1	0	1	0	1	1	1	1	0	1	1	1	0.500	
19	0	1	0	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0	1	0	0	1	0	0	1	0	1	1	1	1	1	1	0.444	
20	1	1	0	1	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0	1	0	0	1	0	0	1	1	1	1	0	1	1	0.556	
21	1	1	1	1	0	1	1	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	0.667	
22	1	1	1	1	1	0	1	1	1	1	1	1	0	1	1	1	0	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	0.500	
23	0	1	1	1	1	1	0	1	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	1	0	0	1	0	1	1	0.556	
24	1	1	0	1	1	1	1	0	1	1	1	1	1	0	1	1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	0	1	0.611
25	1	1	1	0	1	1	1	1	0	1	1	1	1	1	0	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	1	1	0.611	
26	1	1	1	1	0	1	1	1	1	0	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0	0	1	0	1	0	0	0	1	0.556
27	0	1	1	1	1	0	1	1	1	1	0	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0	1	0	1	0	0	1	0	0.667
28	1	1	0	1	1	1	1	0	1	1	1	1	1	1	1	0	1	1	1	1	1	1	0	1	0	1	0	1	0	1	0	0	0	0.556
29	0	1	0	1	0	1	1	1	0	1	1	1	1	1	1	0	1	1	1	1	1	0	0	0	0	0	0	1	0	1	0	1	0	0.667
30	0	0	0	1	0	1	1	1	1	0	1	1	1	1	1	1	0	1	1	1	1	1	0	0	0	0	0	0	1	0	1	0	1	0.556

7) Выведите на график в Matlab функцию автокорреляции в зависимости от величины задержки (lag).



Пик при $\text{lag} = 0$ - максимальная корреляция с самой собой

При $\text{lag} \neq 0$ - значения ниже 1, в диапазоне 0.4–0.7

ВЫВОДЫ

В ходе выполнения я изучила какие существуют псевдослучайные двоичные последовательности, какими корреляционными свойствами они обладают и как используются для синхронизации приемников и передатчиков в сетях мобильной связи.

КОД

Ссылка на github



КОНТРОЛЬНЫЕ ВОПРОСЫ

1) Для чего в мобильных сетях могут использоваться псевдослучайные последовательности?

- Синхронизация приёмника и передатчика.
- Расширение спектра.
- Оценка BER (вероятность битовой ошибки).

2) Что значит положительная корреляция сигналов?

Положительная корреляция – два сигнала ведут себя схожим образом:

- когда один сигнал возрастает, другой тоже имеет тенденцию возрасти, и наоборот

3) Что такое корреляционный прием сигналов?

Корреляционный приём - метод обнаружения и демодуляции сигнала, при котором принятый сигнал сравнивается (коррелируется) с шаблоном (опорной

последовательностью). Если результат корреляции превышает порог - сигнал обнаружен.

- 4) Как вычисление корреляционных функций помогает синхронизироваться приемникам и передатчика в сетях мобильной связи?

Приёмник:

- Последовательно сдвигает локальную копию PN-кода
- Вычисляет корреляцию с принимаемым сигналом на каждом сдвиге
- Находит сдвиг с максимальной корреляцией - это и есть точная временная синхронизация

- 5) Какими свойствами обладают псевдослучайные битовые последовательности?

- Сбалансированность: число единиц и число нулей на любом интервале последовательности должно отличаться не более чем на одну.
- Цикличность: циклом в данном случае является последовательность бит с одинаковыми значениями. В каждом фрагменте псевдослучайной битовой последовательности примерно половину составляли циклы длиной 1, одну четверть – длиной 2, одну восьмую – длиной 3 и т.д.
- Корреляция: корреляция оригинальной битовой последовательности с ее сдвинутой копией должна быть минимальной. Автокорреляция этих последовательностей – практически дельта-функция во временной области, а в частотной области – это константа.

- 6) Какие разновидности PN-последовательностей вам известны?

- M-последовательности
- Коды Голда - XOR двух m-последовательностей
- Коды Касами
- Коды Уолша–Адамара